

CS 2201

Computer Algorithms

Merge Sort

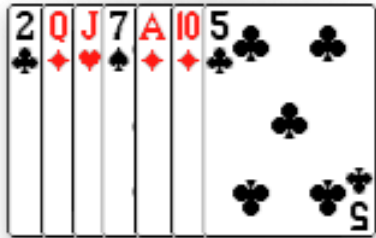
Solving Recurrences

Iteration Method

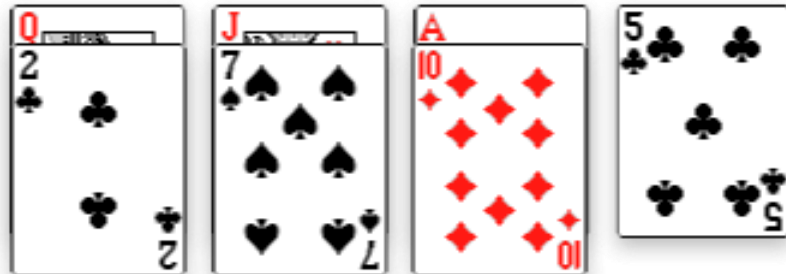
Merge Sort (cont'd)

- Example with a hand of seven cards

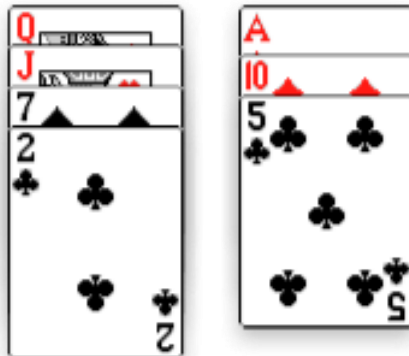
initial hand



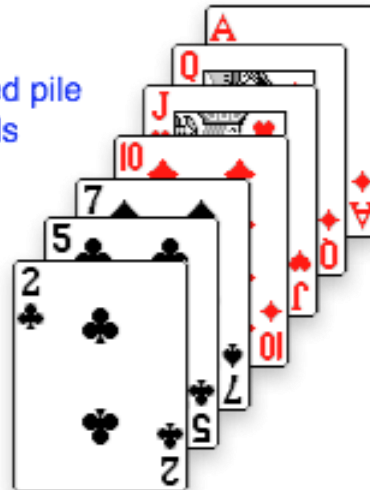
sorted piles of size two



sorted piles of size four



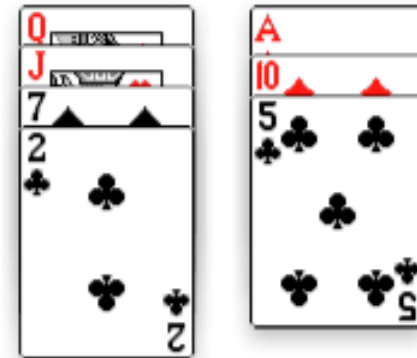
final sorted pile of all cards



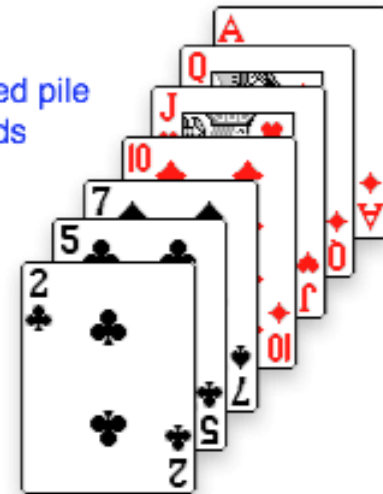
Merge Sort

- What makes this method more effective than simple insertion sort?
 - **merging** two piles is a very simple operation
 - **only need to look at the two cards currently on the top of each pile**
 - no need to look deeper into either group
- In this example:
 - compare 2 with 5, pick up the 2
 - compare 5 with 7, pick up the 5
 -

sorted piles of size four



final sorted pile of all cards



Merge Sort

```
MergeSort(A, left, right) {  
    if (left < right) {  
        mid = floor((left + right) / 2);  
        MergeSort(A, left, mid);  
        MergeSort(A, mid+1, right);  
        Merge(A, left, mid, right);  
    }  
}
```

```
// Merge() takes two sorted subarrays of A and  
// merges them into a single sorted subarray of A  
//      (how long should this take?)
```

Merge Sort: Example

- Show MergeSort() running on the array

A = {10, 5, 7, 6, 1, 4, 8, 3, 2, 9};

Analysis of Merge Sort

Statement	Effort
<code>MergeSort(A, left, right) {</code>	$T(n)$
<code>if (left < right) {</code>	$\Theta(1)$
<code>mid = floor((left + right) / 2);</code>	$\Theta(1)$
<code>MergeSort(A, left, mid);</code>	$T(n/2)$
<code>MergeSort(A, mid+1, right);</code>	$T(n/2)$
<code>Merge(A, left, mid, right);</code>	$\Theta(n)$
<code>}</code>	

- So $T(n) = \begin{cases} \Theta(1) & \text{when } n = 1, \text{ and} \\ 2T(n/2) + \Theta(n) & \text{when } n > 1 \end{cases}$
- So what (more succinctly) is $T(n)$?

Recurrences

- The expression:

$$T(n) = \begin{cases} c & n = 1 \\ 2T\left(\frac{n}{2}\right) + cn & n > 1 \end{cases}$$

is a *recurrence*.

- Recurrence: an equation that describes a function in terms of its value on smaller functions

Recurrence Examples

$$s(n) = \begin{cases} 0 & n = 0 \\ c + s(n-1) & n > 0 \end{cases}$$

$$s(n) = \begin{cases} 0 & n = 0 \\ n + s(n-1) & n > 0 \end{cases}$$

$$T(n) = \begin{cases} c & n = 1 \\ 2T\left(\frac{n}{2}\right) + c & n > 1 \end{cases}$$

$$T(n) = \begin{cases} c & n = 1 \\ aT\left(\frac{n}{b}\right) + cn & n > 1 \end{cases}$$

Solving Recurrences

- Substitution method
- Iteration method
- Master method

Solving Recurrences

- The substitution method (CLR 4.1)
 - A.k.a. the “making a good guess method”
 - Guess the form of the answer, then use induction to find the constants and show that solution works
 - Examples:
 - $T(n) = 2T(n/2) + \Theta(n) \rightarrow T(n) = \Theta(n \lg n)$
 - $T(n) = 2T(\lfloor n/2 \rfloor) + n \rightarrow ???$

Solving Recurrences

- The substitution method (CLR 4.1)
 - A.k.a. the “making a good guess method”
 - Guess the form of the answer, then use induction to find the constants and show that solution works
 - Examples:
 - $T(n) = 2T(n/2) + \Theta(n) \rightarrow T(n) = \Theta(n \lg n)$
 - $T(n) = 2T(\lfloor n/2 \rfloor) + n \rightarrow T(n) = \Theta(n \lg n)$
 - $T(n) = 2T(\lfloor n/2 \rfloor) + 17) + n \rightarrow ???$

Solving Recurrences

- The substitution method (CLR 4.1)
 - A.k.a. the “making a good guess method”
 - Guess the form of the answer, then use induction to find the constants and show that solution works
 - Examples:
 - $T(n) = 2T(n/2) + \Theta(n) \rightarrow T(n) = \Theta(n \lg n)$
 - $T(n) = 2T(\lfloor n/2 \rfloor) + n \rightarrow T(n) = \Theta(n \lg n)$
 - $T(n) = 2T(\lfloor n/2 \rfloor + 17) + n \rightarrow \Theta(n \lg n)$

Solving Recurrences

- Another option is what the book calls the “iteration method”
 - Expand the recurrence
 - Work some algebra to express as a summation
 - Evaluate the summation
- We will show several examples

$$s(n) = \begin{cases} 0 & n = 0 \\ c + s(n-1) & n > 0 \end{cases}$$

- $s(n) =$
 $c + s(n-1)$
 $c + c + s(n-2)$
 $2c + s(n-2)$
 $2c + c + s(n-3)$
 $3c + s(n-3)$
 $\dots$
 $kc + s(n-k) = ck + s(n-k)$

$$s(n) = \begin{cases} 0 & n = 0 \\ c + s(n-1) & n > 0 \end{cases}$$

- So far for $n \geq k$ we have
 - $s(n) = ck + s(n-k)$
- What if $k = n$?
 - $s(n) = cn + s(0) = cn$

$$s(n) = \begin{cases} 0 & n = 0 \\ c + s(n-1) & n > 0 \end{cases}$$

- So far for $n \geq k$ we have

$$- s(n) = ck + s(n-k)$$

- What if $k = n$?

$$- s(n) = cn + s(0) = cn$$

- So $s(n) = \begin{cases} 0 & n = 0 \\ c + s(n-1) & n > 0 \end{cases}$

- Thus in general

$$- s(n) = cn$$

$$s(n) = \begin{cases} 0 & n = 0 \\ n + s(n-1) & n > 0 \end{cases}$$

- $s(n)$

$$= n + s(n-1)$$

$$= n + n-1 + s(n-2)$$

$$= n + n-1 + n-2 + s(n-3)$$

$$= n + n-1 + n-2 + n-3 + s(n-4)$$

$$= \dots$$

$$= n + n-1 + n-2 + n-3 + \dots + n-(k-1) + s(n-k)$$

$$s(n) = \begin{cases} 0 & n = 0 \\ n + s(n-1) & n > 0 \end{cases}$$

- $s(n)$

$$= n + s(n-1)$$

$$= n + n-1 + s(n-2)$$

$$= n + n-1 + n-2 + s(n-3)$$

$$= n + n-1 + n-2 + n-3 + s(n-4)$$

$$= \dots$$

$$= n + n-1 + n-2 + n-3 + \dots + n-(k-1) + s(n-k)$$

$$= \sum_{i=n-k+1}^n i + s(n-k)$$

$$s(n) = \begin{cases} 0 & n = 0 \\ n + s(n-1) & n > 0 \end{cases}$$

- So far for $n \geq k$ we have

$$\sum_{i=n-k+1}^n i + s(n-k)$$

$$s(n) = \begin{cases} 0 & n = 0 \\ n + s(n-1) & n > 0 \end{cases}$$

- So far for $n \geq k$ we have

$$\sum_{i=n-k+1}^n i + s(n-k)$$

- What if $k = n$?

$$s(n) = \begin{cases} 0 & n = 0 \\ n + s(n-1) & n > 0 \end{cases}$$

- So far for $n \geq k$ we have

$$\sum_{i=n-k+1}^n i + s(n-k)$$

- What if $k = n$?

$$\sum_{i=1}^n i + s(0) = \sum_{i=1}^n i + 0 = n \frac{n+1}{2}$$

$$s(n) = \begin{cases} 0 & n = 0 \\ n + s(n-1) & n > 0 \end{cases}$$

- So far for $n \geq k$ we have

$$\sum_{i=n-k+1}^n i + s(n-k)$$

- What if $k = n$?

$$\sum_{i=1}^n i + s(0) = \sum_{i=1}^n i + 0 = n \frac{n+1}{2}$$

- Thus in general

$$s(n) = n \frac{n+1}{2}$$

$$T(n) = \begin{cases} c & n = 1 \\ 2T\left(\frac{n}{2}\right) + c & n > 1 \end{cases}$$

- $T(n) =$
 $2T(n/2) + c$
 $2(2T(n/2/2) + c) + c$
 $2^2T(n/2^2) + 2c + c$
 $2^2(2T(n/2^2/2) + c) + 3c$
 $2^3T(n/2^3) + 4c + 3c$
 $2^3T(n/2^3) + 7c$
 $2^3(2T(n/2^3/2) + c) + 7c$
 $2^4T(n/2^4) + 15c$
 $\dots$
 $2^kT(n/2^k) + (2^k - 1)c$

$$T(n) = \begin{cases} c & n = 1 \\ 2T\left(\frac{n}{2}\right) + c & n > 1 \end{cases}$$

- So far for $n > 2^k$ we have
 - $T(n) = 2^k T(n/2^k) + (2^k - 1)c$
- What if $k = \lg n$?
 - $T(n) = 2^{\lg n} T(n/2^{\lg n}) + (2^{\lg n} - 1)c$
 - $= n T(n/n) + (n - 1)c$
 - $= n T(1) + (n-1)c$
 - $= nc + (n-1)c = (2n - 1)c$

The End